Kartylicious: product specification report

Margarida Pinheiro [125011], Rodrigo Sampaio [125274], Miguel Sousa [125624]
v2026-11-03

1 1 Introduction **1**
1.1 Overview of the project 1
1.2 Known limitations 2
1.3 References and resources 2

2 Product concept and requirements **2**
2.1 Vision statement 2
2.2 Personas and scenarios 2
2.3 Project roadmap and priorities 2

3 Domain model and rules **2**
3.1 Conceptual model 2
3.2 Business rules 3
3.3 Special lifecycles 3

4 Architecture notebook **3**
4.1 Key requirements and constraints 3
4.2 Non-functional requirements and SLOs 3
4.3 Architecture view 4
4.4 Deployment view 4

5 API for developers **4**

[This report is the main source of technical documentation on the project, clarifying the functional scope and architectural choices. Provide concise, but informative content, **allowing other software engineers to understand the product**.
Tips on the expected content placed along the document are meant to be removed!
The report grows with the project.
You may use English or Portuguese; do not mix.]

1 Introduction

1.1 Overview of the project

Tip: project positioning in two or three sentences. Then introduce the product: what it is good for and for whom, and the concretization of the generic theme the team chose.

Kartylicious, It's so delicious! YUMMY!

Kartylicious is a managed fleet-as-a-service of light electric mini-karts for everyday urban trips. It turns short journeys into something fun: a stylish alternative to e-scooters and shared bikes.

It's the ideal way to get around the city if you want a fun ride. If bicycles and e-scooters aren't your first choice, mini-karts are your go-to: take a seat and enjoy a comfortable ride to your destination, alone, with a group, or even on a date.

Our concretization follows the Municipal concession model, set in a fictional city with wide, accessible pavements. The mini-kart is defined as a light electric vehicle, in the same category as an e-scooter: a top speed of 25 km/h, no registration, and use only in zones defined by the city. All names (operator, client and city) are fictional.

1.2 Known limitations

Tip: List features that were planned or expected but not implemented, and why. Review and complete this section by the end of the project.

1.3 References and resources

Tip: List the key components (libraries, web services, standards such as GBFS) and references (articles, tutorials) that really helped and would help other students doing similar work.

2 Product concept and requirements

2.1 Vision statement

Tip: Give a functional (black-box) description of the product. Which business problem does it solve? Which key features address it? Optionally, compare it with well-known products and explain how the concept was developed and who helped with the requirements. If the concept changed during the project, say what changed and why.

Kartylicious is a managed light-mobility fleet-as-a-service platform that transforms short urban trips into safe, comfortable, and engaging journeys while solving municipal transit governance challenges. Addressing both user hesitation toward standing e-scooters and the regulatory concerts of municipal councils, the platform provides a seamless hub-to-hub rental experience for street legal electric mini-karts. Through an end user interface, Kartylicious delivers vehicle

discovery, booking, and payment alongside critical administrative features: real-time telemetry monitoring and transparent SLA tracking for the municipal concession partner.

2.2 Personas and scenarios

Tip: Personas are fictional people with names, occupations, goals and life stories; they make design, development and testing more emphatic (Grudin and Pruitt, 2002). Create one persona per main actor (end user, operator, client manager) and see section 4.1 of this article for related samples.
Write one or more scenarios per persona, focused on the core value of the system. A scenario tells a story with a goal, like a use case, but covers a larger process that may include activities outside the software; it needs no fixed template. See section 4.2 of the same article.

2.3 Project roadmap and priorities

Tip: Give an indicative plan for building the product incrementally over the iterations (I1–I5), naming the epics and the features each iteration delivers.
Tip: include a *roadmap\timeline* view (from Jira).

3 Domain model and rules

3.1 Conceptual model

Tip: Which information concepts does the product manage, and how are they related? Use a **logical model** (UML class diagram) with the concepts and their attributes, not an entity-relationship database model.

3.2 Business rules

Tip: List the essential business rules: the constraints and calculations the product must enforce, whatever the interface. Give each rule an ID, a precise statement with its parameters, and the tests that cover it. State boundary values explicitly.
Rules connect the acceptance criteria (2.5) to the tests: reference rule IDs in Gherkin scenarios (for example, as tags such as @BR3) and in Xray. The rows below are examples for a campus e-bike service; replace them with yours.

ID	Rule	Parameters and boundaries	Covered by
BR1	An e-bike can be booked only if it is available and its battery is at or above the minimum level	Minimum 20 %: 19 % refused, 20 % accepted	Unit tests; Gherkin @BR1
BR2			
BR3			

ID	Rule	Parameters and boundaries	Covered by

3.3 Special lifecycles

State machines worth noticing. Include a state diagram of the vehicle lifecycle (available, reserved, in use, maintenance, retired) with its transitions: it drives both the business logic and its tests.

4 Architecture notebook

4.1 Key requirements and constraints

Tip: Identify the issues that drive the architecture. Examples: integration with external systems (payment gateway, fleet simulator, open-data consumers); a telemetry stream that keeps arriving while users book; several user-facing interfaces; data that must stay consistent when two users try to book the same vehicle.
For a systematic approach, review the catalogue of architectural characteristics and identify those relevant to your project; they drive the key design decisions and the non-functional tests. (O'Reilly Learning; requires institutional access.)

4.2 Non-functional requirements and SLOs

Tip: State the Service Level Objectives the team commits to, by I2 at the latest. Each SLO needs a measurable target and the test that checks it. Example rows below; replace them with yours.

ID	Objective	Metric and target	Verified by
SLO1	Booking is fast under normal load	p95 latency of POST /bookings < 500 ms with 50 concurrent users	k6 test with thresholds
SLO2	Telemetry is processed promptly	Lag between reading and vehicle state update < 5 s at 10 readings/s	k6 test or simulator run
SLO3			

4.3 Architecture view

System architecture
Tip: Present the planned architecture: the main building blocks and how they interact. Include a diagram (a logical view, such as a package or block diagram). Avoid implementation technologies and deployment details here; protocols and standards can appear. Name the architecture style applied, if any.

Modules interaction
Use a sequence diagram to show core flows across the system, for example: check-in, then telemetry update, then end of use and charge.

External integrations
For each integration (fleet simulator, payment sandbox, identity provider, GBFS feed), state the interface, the protocol, the adapter that isolates it and how it is tested. The fleet simulator must sit behind one port and adapter, so a simulator provided by the teaching team can replace the team's own (guidelines, section 4.4).

4.4 Deployment view

Tip: Explain the production configuration: containers, hosts, IP addresses and ports, databases, the simulator, environments (staging, production). Show the implementation technologies in the diagram, for example the PostgreSQL symbol on the database. Complete this view when the deployments are in place.

5 API for developers

Tip: Explain how the API is organized and its main resource collections, in general terms. Document the methods in a hosted solution (for example, OpenAPI with Swagger UI) and link it here and in the README.
Follow best practices for REST API design: a REST API is designed around resources, the key entities the application exposes (vehicles, hubs, bookings), not around actions. If the team publishes a GBFS feed, link it here too.